

TECNICATURA SUPERIOR EN DESARROLLO DE SOFTWARE – IES TINOGASTA

Modulo: SEMINARIO

Clausulas para la presentación de la propuesta de Desarrollo de Software

Presentación: Individual

Ciclo de vida del desarrollo de software



¿Cómo crear un proyecto de desarrollo de software exitoso?

Un proyecto de desarrollo de software puede ser una tarea compleja si no se toman en cuenta aspectos clave desde el inicio. Para que un software sea funcional, escalable y cumpla con los objetivos esperados, es necesario definir sus requerimientos, elegir la metodología correcta y seguir buenas prácticas en su desarrollo.

Conocer los elementos fundamentales del desarrollo te permitirá optimizar recursos y evitar errores comunes. En este post, conocerás todos los pasos necesarios para ayudarte a llevar a cabo un proyecto de software exitoso.

Qué es el desarrollo de software

El desarrollo de software es el proceso de crear programas y aplicaciones que permiten a los usuarios realizar diversas tareas en computadoras, dispositivos móviles o sistemas empresariales. Su objetivo principal es diseñar soluciones

tecnológicas que optimicen procesos y resuelvan problemas en diferentes sectores.

Este proceso no se limita a la programación, sino que también abarca decisiones estratégicas sobre:

- **Arquitectura del software:** define la estructura y organización del sistema.
- **Lenguaje de programación:** selección del código adecuado según los requerimientos del proyecto.
- **Enfoque metodológico:** elección de técnicas y procesos de desarrollo para optimizar tiempos y recursos.
- **Planificación estratégica:** establecimiento de objetivos y recursos antes de iniciar el proyecto.

Cada una de estas decisiones influye en las características del software final, así como su calidad y funcionalidad, por lo que una planificación adecuada es clave para el éxito de cualquier proyecto de desarrollo de software.

Cómo planificar un proyecto de desarrollo de software

Planificar un proyecto de desarrollo de software contribuye para evitar retrasos, sobrecostos y problemas técnicos. Una buena planificación permite organizar el trabajo, definir objetivos claros y asegurar que el software cumpla con las expectativas del usuario.

Cada decisión tomada en esta etapa influirá en la calidad y sostenibilidad del software a largo plazo. Estos son los factores que se deben considerar para una planificación efectiva:

1. Define los requisitos (Primera presentación: Aprobar la idea del proyecto elegido)

El primer paso en un proyecto de desarrollo de software es definir claramente sus requisitos. Esto implica identificar las necesidades del usuario, los objetivos del sistema y las funcionalidades esenciales, ya que una mala definición en esta etapa puede generar errores costosos en fases posteriores.

Para estructurar bien los requisitos, es recomendable:

- **Recopilación de información:** realizar entrevistas con clientes y usuarios finales para comprender sus necesidades.

- **Documentación detallada:** registrar funcionalidades clave, restricciones y objetivos del software.
- **Priorización de características:** clasificar las funciones según su importancia y viabilidad dentro del proyecto.
- **Uso de herramientas visuales:** aplicar historias de usuario, diagramas de flujo o mapas conceptuales para representar los procesos del sistema.
- **Realiza un relato de los requerimientos y la solución:** de toda la información recabada narra los detalles de los requerimientos del software y propone una solución posible.

2. Selecciona la metodología de desarrollo (Segunda presentación con la elección de la metodología a usar con su justificación)

Elegir **la metodología de desarrollo de software** adecuada ayuda a definir la estructura del trabajo y la forma en que se llevarán a cabo las distintas etapas. La elección dependerá de factores como la complejidad del proyecto, el presupuesto, los plazos y la flexibilidad requerida. Entre las más usadas están:

- **Waterfall (Cascada):** desarrollo en fases secuenciales donde cada etapa debe completarse antes de avanzar a la siguiente. Es ideal cuando los requisitos están bien definidos desde el inicio y no se esperan grandes cambios.
- **Metodología de Prototipo:** permite construir versiones preliminares del software para recibir retroalimentación de los usuarios antes del desarrollo final. Es útil cuando los requerimientos no están completamente definidos.
- **Modelo Incremental:** divide el desarrollo en fases, agregando nuevas funcionalidades en cada ciclo. Cada módulo se prueba antes de completar el sistema.
- **Metodología en Espiral:** combina desarrollo iterativo con análisis de riesgos. Se realizan mejoras continuas basadas en la retroalimentación del cliente.
- **Diseño Rápido de Aplicaciones (RAD):** prioriza el desarrollo ágil mediante la creación de prototipos funcionales y retroalimentación constante. Es ideal para proyectos que requieren entregas rápidas.

3. Diseña el sistema (Tercera presentación: en esta etapa se presentan las plataformas, arquitectura de hardware y software, base de datos, frontend y backend, informes y reportes, seguridad y backup.)

El diseño del sistema define la arquitectura y estructura del programa antes de la implementación. Un buen diseño facilita la escalabilidad, mantenimiento y eficiencia del software. En esta etapa se consideran aspectos como:

- **Arquitectura del software:** define la organización y comunicación entre los componentes del sistema. Puede ser monolítica, donde todo el software se ejecuta en una sola unidad, o basada en microservicios, lo que permite mayor flexibilidad y escalabilidad.
- **Base de datos:** se debe elegir entre bases de datos relacionales (SQL), ideales para datos estructurados, o NoSQL, recomendadas para grandes volúmenes de datos no estructurados. La organización de la información influye en la velocidad y eficiencia del sistema.
- **Interfaces y experiencia de usuario (UI/UX):** un diseño intuitivo mejora la interacción del usuario con el software. Se deben considerar aspectos como navegación fluida, accesibilidad y compatibilidad con distintos dispositivos.
- **Seguridad:** se deben aplicar protocolos de seguridad como encriptación de datos, autenticación de usuarios y protección contra ciberataques, asegurando la integridad del software y la privacidad de la información.

4. Desarrolla el software (Cuarta presentación: consultas periódicas de los avances del proyecto. Diseño de interfaces, código de programación, flujo de datos y reportes)

El desarrollo es la fase en la que se construye el software según los requerimientos y el diseño establecido. En este proceso, los programadores escriben el código utilizando los lenguajes de programación, frameworks y herramientas adecuadas.

Para garantizar un desarrollo eficiente y de calidad, se deben seguir buenas prácticas como:

- **Código limpio y estructurado:** facilita el mantenimiento y futuras modificaciones.
- **Uso de control de versiones (Git, GitHub, GitLab):** permite gestionar cambios y colaborar en equipo.
- **Pruebas unitarias:** aseguran que cada módulo funcione correctamente antes de la integración.
- **Metodologías ágiles:** permiten desarrollar en ciclos iterativos, lo que facilita adaptarse a cambios y optimizar el software continuamente.

5. Implementa el software (Quinta presentación: carga de datos a la base, prueba de código abierto y manejo de errores)

La implementación es el proceso de poner en funcionamiento el software desarrollado, asegurando que pueda ser utilizado de manera efectiva por los usuarios finales. Esta fase debe ejecutarse con planificación para evitar fallos críticos y garantizar una transición fluida.

Los principales enfoques de implementación incluyen:

- **Implementación en entornos de prueba:** se despliega el software en un ambiente controlado para identificar posibles errores antes del lanzamiento final.
- **Despliegue en servidores o la nube:** se aloja el software en un entorno accesible, garantizando disponibilidad y escalabilidad.
- **Monitoreo post-lanzamiento:** se evalúa el rendimiento del software en tiempo real para detectar fallos y optimizar el sistema.
- **Capacitación a usuarios:** en algunos casos, se debe entrenar a los usuarios para garantizar un uso eficiente del software.

6. Ejecuta pruebas y control de calidad (Sexta presentación: Pruebas de funcionamiento del software en distintos escenarios)

Antes de que un proyecto de desarrollo de software sea lanzado, es necesario realizar pruebas para detectar errores y garantizar su correcto funcionamiento. Las pruebas permiten evaluar el rendimiento, la seguridad y la usabilidad del sistema, asegurando que cumpla con los requisitos establecidos. Existen diferentes tipos de pruebas en esta etapa:

- **Pruebas unitarias:** verifican que cada módulo o función del software opere correctamente de forma individual.
- **Pruebas de integración:** evalúan cómo interactúan entre sí los distintos componentes del sistema.
- **Pruebas de rendimiento:** miden la velocidad y estabilidad del software bajo diferentes condiciones de uso.
- **Pruebas de seguridad:** identifican vulnerabilidades para evitar riesgos como accesos no autorizados o pérdida de datos.
- **Pruebas de aceptación del usuario (UAT):** validan si el software cumple con las expectativas y necesidades del cliente final.

7. Realiza mantenimiento y actualizaciones (Séptima presentación: realizan pruebas de mantenimiento y reportes de información)

El mantenimiento es una fase continua en el desarrollo de software, ya que ningún sistema es completamente estático. Con el tiempo, surgen errores, cambios en los requerimientos o la necesidad de optimizar el rendimiento del software. Uno de los **servicios informáticos** más importantes es el mantenimiento:

- **Mantenimiento correctivo:** soluciona errores o fallos detectados después del lanzamiento.
- **Mantenimiento adaptativo:** ajusta el software para que funcione en nuevos entornos o plataformas.
- **Mantenimiento perfectivo:** mejora el rendimiento o agrega nuevas funcionalidades según las necesidades del usuario.
- **Mantenimiento preventivo:** busca detectar y corregir posibles problemas antes de que afecten el funcionamiento del sistema.

8. Mantén todo documentado (Presentación FINAL: Carpeta con la documentación del proyecto y el software ejecutable multiplataforma para su defensa)

Mantener una documentación clara y actualizada evita problemas cuando el software necesite modificaciones o mantenimiento a largo plazo. Además,

facilita la transferencia de conocimiento dentro del equipo de desarrollo. Estos son algunos tipos de documentación que deben considerarse:

- **Documentación técnica:** detalla la arquitectura, código fuente, API y bases de datos, permitiendo a los desarrolladores futuros comprender y modificar el software.
- **Documentación del usuario:** manuales y guías que explican cómo utilizar el software de manera eficiente.
- **Registros de cambios (changelog):** historial de actualizaciones, mejoras y correcciones aplicadas al sistema.